Software Requirements Specification CSC4351 Capstone 1

Team #11
Members: Dominque Hogan, Leilani Leon, Parker Mills, Ticiana Ward
Section: Capstone Tuesday Section

Table of Contents

Table of Contents 1
Introduction / Overview 2
General description 2
Functional Requirements 2
Interface Requirements 3
    User Interface (UI): 3
    Software Interfaces: 3
    External System Interfaces 4
Performance Requirements 4
Design Constraints 4
Non-Functional Requirements 4
Appendices 5
    Acronyms and Definitions 5

Introduction / Overview

The Text Corpus Management System (TCMS) is an agent-based corpus analysis tool. This innovative software system leverages Large Language Models (LLMs) and the LangChain framework to automate complex linguistic analysis tasks on text corpora. This system represents a shift from traditional corpus linguistics tools by providing an intelligent, agent-based architecture where specialized software agents autonomously perform linguistic analyses through natural language interaction.

The software enables researchers, linguists, educators, and students to analyze large collections of text without requiring extensive programming expertise. By integrating cutting-edge AI technologies with established computational linguistics methods, the system bridges the gap between symbolic and non-symbolic approaches to language analysis.

General description

The TCMS will be a flexible, agent-based corpus analysis toolchain designed for linguists, researches, and data scientists to efficiently analyze text corpora without extensive programming expertise. Leveraging Large Language Models (LLMs) and the LangChain framework, the system automates core corpus linguistics tasks such as text preprocessing, frequency analysis, n-gram and collocation extraction, concordance (KWIC) generation, and keyword analysis. Specialized agents handle each task, while a coordinating LLM agent interprets natural language queries, orchestrates the agents, dynamically generates and executes Python code, and integrates results into a comprehensible output. By enabling retrieval-augmented generation (RAG) for conceptual document searches and providing a web-based interface for user interaction, the system ensures automation, flexibility, explainability, and extensibility, supporting advanced linguistic analysis across various registers and text sources.

Functional Requirements

Data Access Agent

- Input: Text corpora from files (TXT, CSV, XML, JSON), databases (SQL), or web sources.
- Process: Read, normalize (UTF-8), segment into sentences, tokenize.
- Output: Standardized, preprocessed corpus suitable for downstream agents.

Frequency Analysis Agent

- Input: Preprocessed text from the Data Access Agent.
- Process: Compute word frequency distributions, generate word lists, apply stopword filtering.
- Output: Frequency tables to identify common patterns or themes.

Concordance (KWIC) Agent

- Input: Preprocessed text corpus + user-specified keyword.
- Process: Generate context windows for keywords; dynamically generate Python code if needed.
- Output: KWIC tables showing occurrences of keywords in context.

N-gram and Collocation Analysis Agent

- Input: Preprocessed text corpus.
- Process: Compute bigrams/trigrams; evaluate collocations using statistical measures such as PMI or Dice coefficient.
- Output: Ranked collocation list with statistical scores.

Keyword Analysis Agent

- Input: Target corpus + reference corpus.
- Process: Compare corpora using log-likelihood, TF-IDF, or other statistical tests to identify significant keywords.
- Output: Keyword lists with significance values.

Coordinating LLM Agent

- Input: User natural language query
- Process: Interpret user query using LLM, map query to relevant agent(s), generate/execute Python code using NLTK, pandas, scipy, aggregate results from multiple agents.
- Output: Processed results including tables, charts, and textual interpretation.

Validation (WatchDog) Agent

- Input: Outputs from all analysis agents (Frequency Analysis, KWIC, Collocation, Keyword Analysis, RAG)
- Process: Check correctness and consistency of results produced by other agents, detect anomalies, missing data, or inconsistencies in outputs, and flag errors or automatically trigger re-execution of affected tasks.
- Output: Validation report indicating verified results, inconsistencies, and corrective actions.

Interface Requirements

User Interface (UI):

- A simple, web-based interface built with frameworks like Flask, Django, or Streamlit.
- Input field for queries in natural language.
- Options for corpus upload, preprocessing configuration, and visualization.

Software Interfaces:

- Each cross-agent communication via Python objects, JSON messages, or LangChain-defined protocols.
- The Coordinating LLM Agent controls multi-agent workflows, ensuring correct sequencing, input/output mapping, and error handling.
- Dynamically generated Python code by the Coordinating Agent executes in a containerized environment.

External System Interfaces

- Databases: SQL or NoSQL systems for storing corpora and analysis outputs.
- Web Sources: Agents may fetch texts via HTTP requests, REST APIs, or web scraping.
- Vector Databases: Semantic search in Retrieval-Augmented Generation (RAG).

Performance Requirements

- The system must support up to 1 million documents per corpus while maintaining within reasonable time
- Handle corpora with millions of tokens
- Optimize vector database searches for large corpora

Design Constraints

- The system must run in Python 3.10+, leveraging libraries such as NLTK, spaCy, pandas, and SciPy for NLP tasks.
- The system must integrate with the LangChain framework for agent orchestration.
- Dynamically generated Python code must execute in a contained environment to prevent unauthorized system access and ensure security.
- Within technical limitations, the agents must process large text corpora efficiently without significant delays
- The data quality and integrity must correctly identify languages in diverse text samples
- With development constraints, the project must be completed within a specific timeline, influencing the scope and complexity of features

Non-Functional Requirements

- Some of the non-functional requirements in the project ensure that the system meets the quality standards necessary for successful deployment and user satisfaction.
- Security:
    - Execute all dynamically generated code in isolated Docker containers
    - Implement code validation to prevent malicious operations

- Reliability:
    - Validate all statistical computations
    - Implement error handling and recovery
    - Provide confidence scores or uncertainty measures
    - Compare results against established tools (e.g., AntConc) during testing

- Maintainability:
    - Modular architecture for easy agent addition/modification
    - Well-documented codebase

- Scalability:
    - Support growing corpus sizes
    - Handle increasing number of concurrent users

Appendices

Acronyms and Definitions

- NLP (Natural Language Processing): A field of artificial intelligence focused on the interaction between computers and human language.
- LLM (Large Language Model): A machine learning model trained on vast amounts of text to perform tasks such as text generation, summarization, and question answering.
- RAG (Retrieval-Augmented Generation): An architecture for optimizing the performance of an artificial intelligence (AI) model by connecting it with external knowledge bases
- KWIC (Keywords in Context): a database search in which the keyword is shown highlighted in the middle of the display, with the text forming its context on either side.
- PMI (Pointwise Mutual Information): a measure of association. It compares the probability of two events occurring together to what this probability would be if the events were independent.
- API (Application Programming Interface): A set of rules and protocols for building and interacting with software applications.
- JSON (JavaScript Object Notation): A lightweight, text-based data format commonly used for transmitting structured data.
- CSV (Comma-Separated Values): A plain-text format for representing tabular data, where fields are separated by commas.
- XML (Extensible Markup Language): A markup language used for encoding documents in a format that is both human-readable and machine-readable.
- PEP8: The official style guide for Python code, providing conventions for readability, consistency, and maintainability.